Complete Fraud Detection Report
Real-Time Banking Transaction Fraud Detection and Decision Support System

Repository: `Final_Project_DMM501_Group1`

April 2026

Contents

1 Executive Summary

This project implements a fraud detection system that combines machine learning inference with operational decision support for transaction handling. The implemented system accepts a transaction feature vector through a FastAPI service, computes an uncalibrated `risk_score`, maps that score to a `risk_tier` and `decision_recommendation`, and creates an `alert` and `case` for suspicious transactions. The broader system also includes an analyst-facing frontend, case lifecycle management, Prometheus metrics, Grafana dashboards, MLflow runtime tracking, automated tests, and Docker-based deployment components.

The key capability is not only fraud scoring, but the conversion of model output into actionable workflow states. In the implemented design, the machine learning layer acts as a ranking mechanism, while operational services determine whether a transaction should be allowed, sent for manual review, challenged through step-up authentication, held, or blocked. This architecture supports analyst triage, queue management, and investigation tracking rather than treating model output as a final adjudication.

The business objective is to reduce fraud loss, control review workload, and limit unnecessary customer friction. It does so by separating low-risk transactions from review-queue transactions and high-risk transactions, while preserving traceability through `alert`, `case`, and timeline records. The implementation is strongest as a local, end-to-end fraud decision-support demonstration and MLOps reference system, rather than as a production-grade banking control stack.

2 Introduction

Fraud detection remains a central operational problem in banking because digital transactions are high volume, time sensitive, and exposed to multiple attack patterns, including unauthorized card use, account takeover, mule activity, and anomalous transfer behavior. Financial institutions must detect suspicious activity quickly enough to contain loss, while also avoiding unnecessary interruption to legitimate customers.

Traditional rule-based fraud systems remain useful, but they have well-known limitations. Static rules can be brittle, generate excessive false positives, and require substantial manual maintenance as fraud patterns evolve. Rules are also difficult to optimize when risk depends on interactions among many variables rather than a small number of predefined conditions.

Machine learning is therefore useful because it can rank transactions by relative suspiciousness from historical patterns. However, in banking operations, model output alone is insufficient. A practical fraud detection system must integrate scoring with policy decisions, analyst queues, case handling, monitoring, and auditability. This repository reflects that broader framing by combining a supervised fraud model with decision logic, `alert` generation, `case` workflow, and operational interfaces.

3 Problem Definition

The core analytical problem is transaction fraud detection on a highly imbalanced dataset. The repository uses the Kaggle credit card fraud dataset, where fraudulent transactions represent a very small minority of all observations. In such settings, standard accuracy is not meaningful because a naive model can predict the majority class and still appear strong.

The operational problem is more complex than binary classification. A false negative can produce direct financial loss, reputational damage, and delayed investigation. A false positive

can create customer friction, reduce transaction completion, and overload analysts with low-value investigations. For this reason, the implemented system converts model output into `risk_tier` values and downstream `decision_recommendation` values rather than returning only a class label.

The project therefore addresses a decision-support problem: rank transactions by risk, separate suspicious from non-suspicious activity, allocate review capacity using explicit thresholds, and support investigation through `alert` and `case` management.

4 Business Context

Banking fraud controls operate under competing constraints. First, fraud loss matters because delayed intervention can allow rapid cash-out or repeated unauthorized transactions. Second, customer friction matters because unnecessary holds, blocks, or manual reviews can cause legitimate payment failure and dissatisfaction. Third, analyst workload matters because review teams have limited capacity and cannot investigate every transaction manually.

The implemented threshold policy in this repository reflects those constraints. Thresholds are selected using a top-K review-rate policy rather than a single generic cutoff. This means the system is designed to approximate operational review capacity by flagging only the highest-scoring fraction of transactions for review and a smaller fraction for high-risk handling.

The project also acknowledges operational realities beyond scoring. A suspicious transaction must enter an `alert` queue, become a `case`, move through status transitions, and eventually reach a resolution such as `CONFIRMED_FRAUD` or `FALSE_POSITIVE`. Monitoring must then track queue size, status transitions, and false-positive outcomes.

5 System Overview

The implemented system follows the pipeline below:

$$\text{Transaction} \rightarrow \text{Validation} \rightarrow \text{Feature processing} \rightarrow \text{Risk scoring} \rightarrow \text{Decision policy} \rightarrow \text{Alert generation} \rightarrow \text{Case creation} \rightarrow \text{Investigation} \rightarrow \text{Resolution}$$

1. Transaction intake through `features` or `features_by_name`, with optional `transaction_id`, `timestamp`, `amount`, `channel`, and `metadata`.
2. Validation of schema consistency, numeric finiteness, and feature length against model metadata.
3. Feature processing for inference.
4. Risk scoring through the deployed model to obtain an uncalibrated `risk_score`.
5. Decision policy mapping to `risk_tier` and `decision_recommendation`.
6. Reason-code generation through a heuristic decision-support layer.
7. `Alert` and `case` creation for `REVIEW` and `HIGH` outcomes.
8. Analyst investigation through queue, case detail, status changes, and timeline.
9. Resolution through final case outcomes.

6 Dataset Description

The repository uses the Kaggle credit card fraud dataset stored locally as `data/archive/creditcard.csv` and `data/raw/creditcard.csv`. According to the generated repository artifacts, the dataset contains 284,807 rows and 31 columns, consisting of `Time`, anonymized variables `V1` through `V28`, `Amount`, and the target variable `Class`.

Fraud is extremely rare in the data. The generated class distribution report shows 492 fraud records and 284,315 non-fraud records, giving a fraud ratio of 0.00172485630620034, or approximately 0.17%.

The feature set has important limitations for banking interpretation. Most predictors are PCA-transformed variables rather than business-readable fields such as merchant category, destination account, device reputation, customer tenure, or beneficiary risk.

6.1 Dataset figures

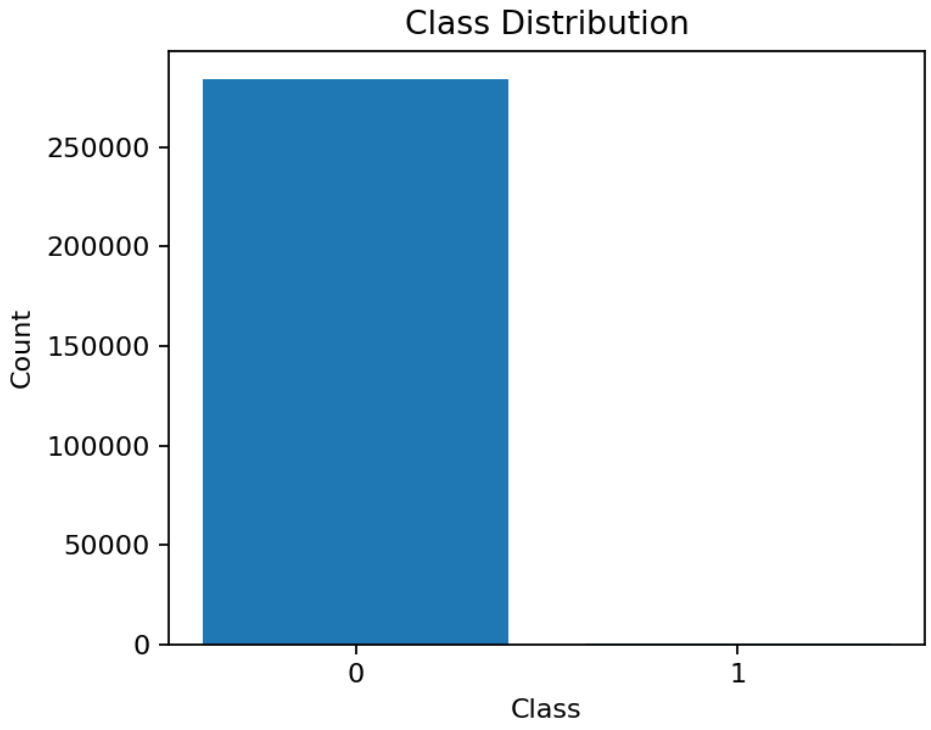


Figure 1: Class imbalance in the benchmark dataset (fraud vs non-fraud).

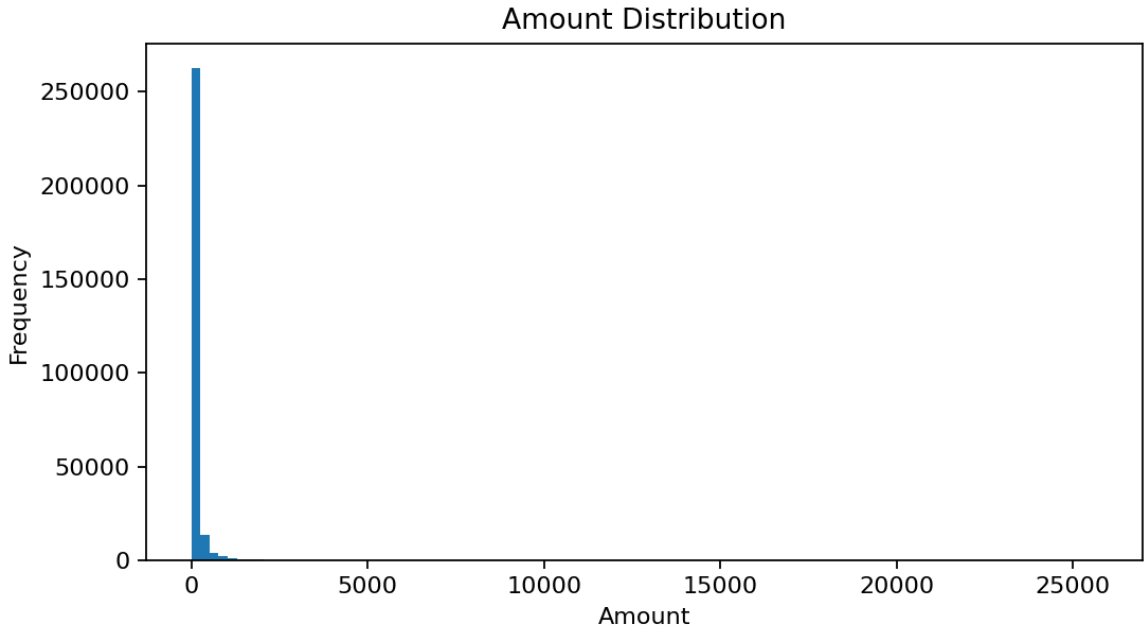


Figure 2: Transaction amount distribution.

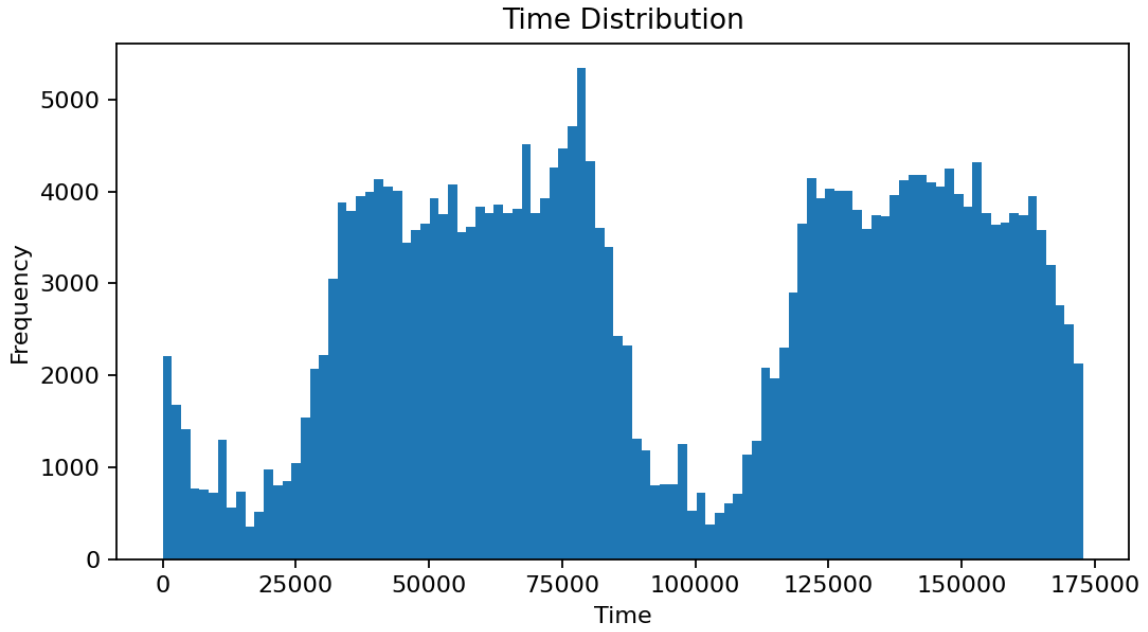


Figure 3: Transaction time distribution.

7 Data Challenges

The first data challenge is extreme class imbalance. Because fraud is rare, a classifier can produce deceptively strong aggregate metrics while still missing many fraudulent transactions or generating

too many false alerts. This is why the repository emphasizes PR-AUC, threshold tuning, and operating-point metrics instead of generic accuracy.

The second challenge is interpretability. The PCA-style `V1`–`V28` variables are mathematically useful but operationally opaque. The implemented reason codes therefore rely partly on policy thresholds and optional metadata heuristics rather than on inherently interpretable core features.

The third challenge is simulation realism. Although the project is framed as a banking fraud detection system, the underlying dataset does not contain the full operational context of a bank.

8 Machine Learning Pipeline

The repository contains an explicit training and model-selection workflow in `src/pipelines/run_model_workflow.py` and `src/pipelines/train_pipeline.py`. The workflow reads the credit card dataset, performs exploratory checks and schema validation, splits data into train, validation, and test subsets, and benchmarks multiple models.

The implemented workflow includes preprocessing, model training, evaluation, threshold tuning, and artifact saving. The saved deployment artifact identifies the selected model as a logistic regression pipeline. The threshold policy is explicitly documented in `artifacts/models/model_info.json` as a capacity-driven top-K approach with `review_top_rate=0.01` and `high_top_rate=0.002`.

9 Model Evaluation

For fraud detection, PR-AUC is more informative than ROC-AUC because the positive class is extremely rare. ROC-AUC can remain high even when a model produces many false positives at the operating point that matters to analysts. PR-AUC focuses more directly on the precision-recall trade-off under class imbalance.

9.1 Model comparison

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC	Status
Logistic Regression (Review)	0.7391	0.1470	0.8510	0.2510	0.9684	0.6300	Selected in de
LightGBM (Review)	0.0000004	0.1330	0.7700	0.2270	0.9288	0.6290	Benchmarked

Table 1: Validation operating-point comparison from repository benchmark artifacts.

The repository also stores the final deployed evaluation metrics at the operating thresholds used for decision support. At the review threshold, precision is 0.1462 and recall is 0.8514. At the high threshold, precision is 0.8429 and recall is 0.7973. These figures show the trade-off between broader queue capture at the review threshold and tighter containment at the high threshold.

9.2 Model evaluation figures

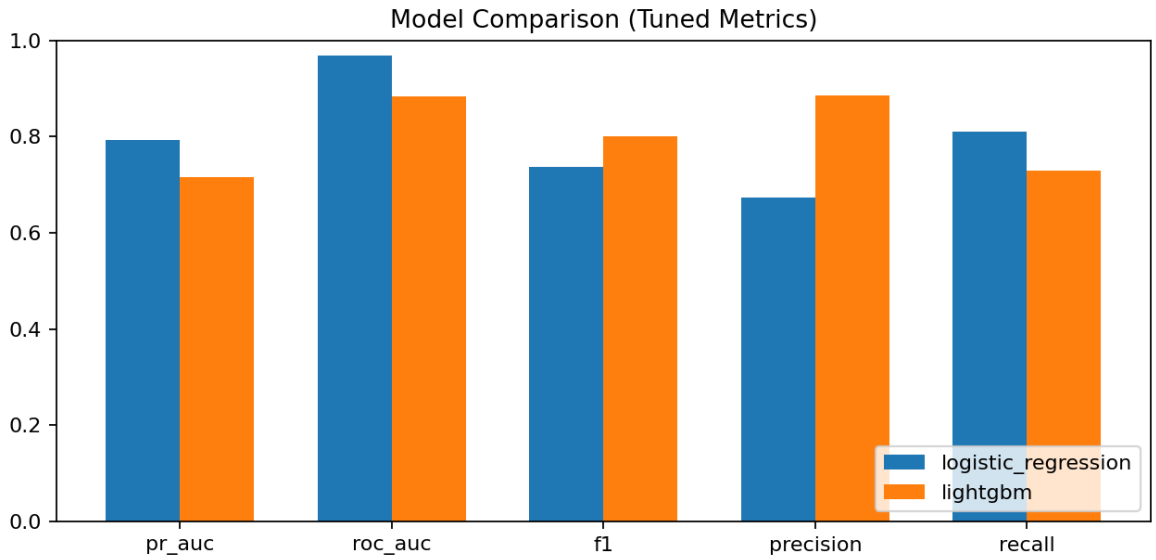


Figure 4: Model benchmark comparison summary from saved artifacts.

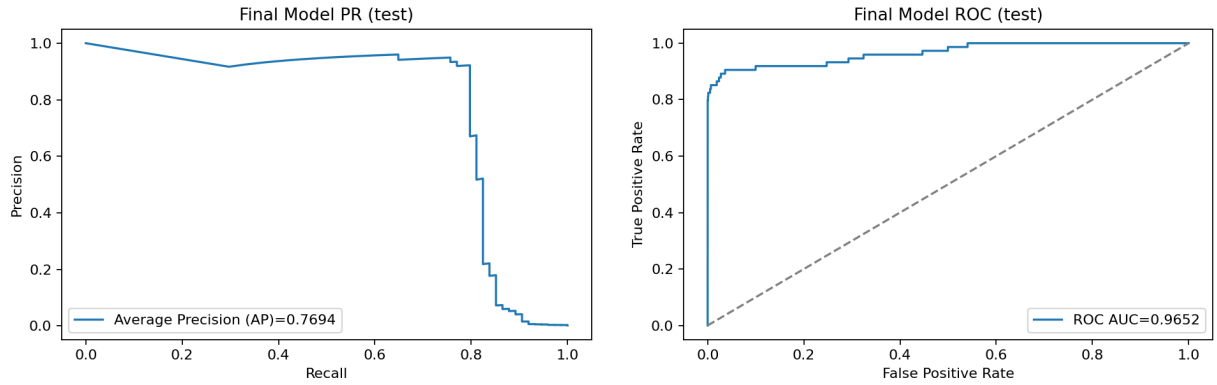


Figure 5: Final deployed model PR curve (left) and ROC curve (right).

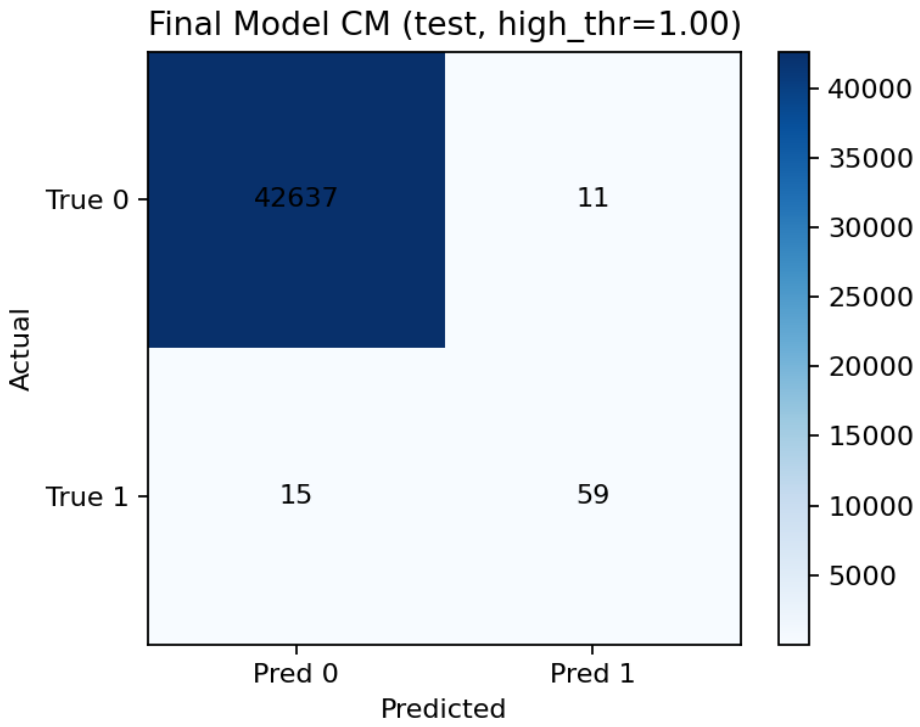


Figure 6: Final deployed model confusion matrix at the selected operating point.

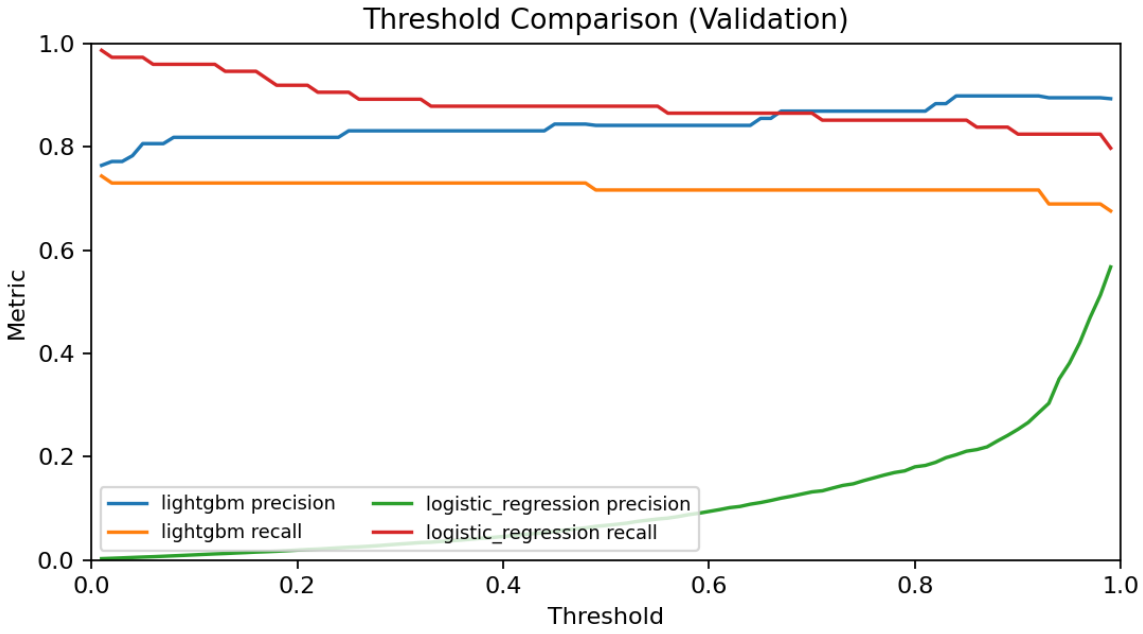


Figure 7: Threshold comparison evidence supporting review/high policy tiers.

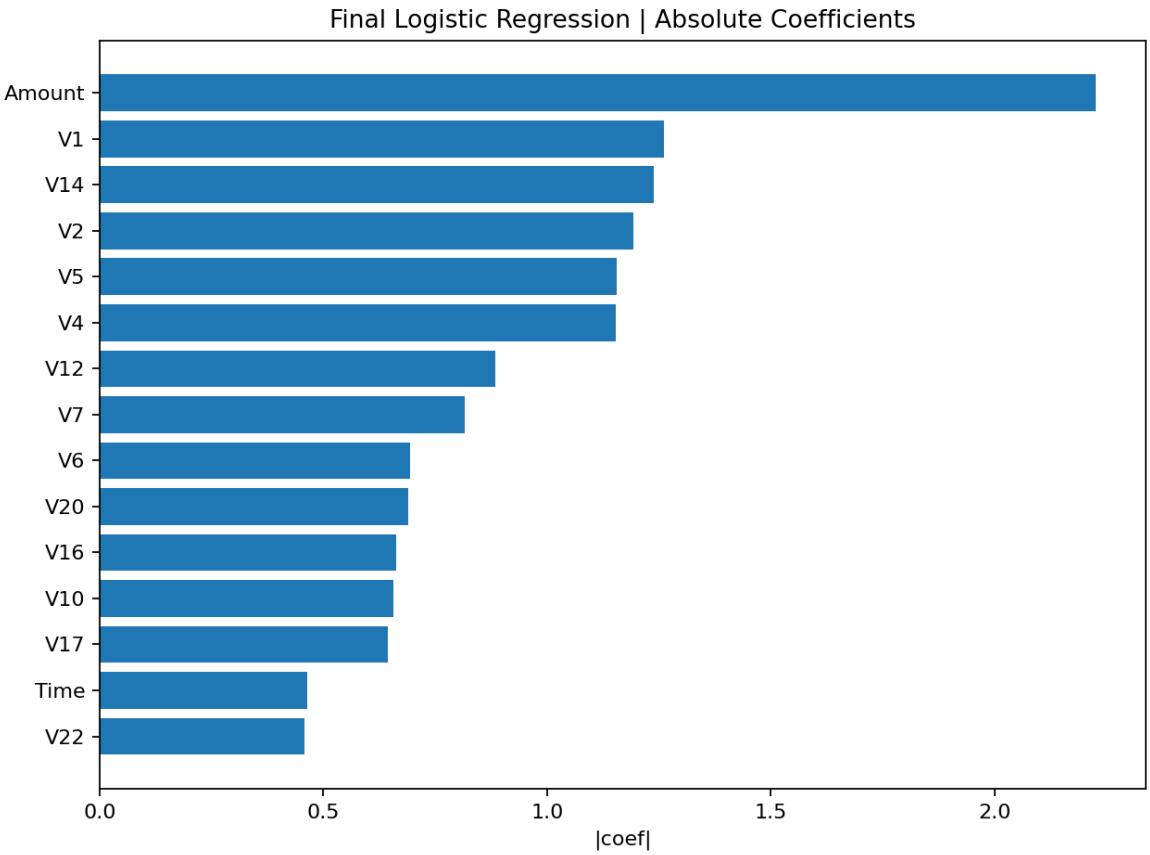


Figure 8: Feature importance evidence from saved artifacts (model-dependent).

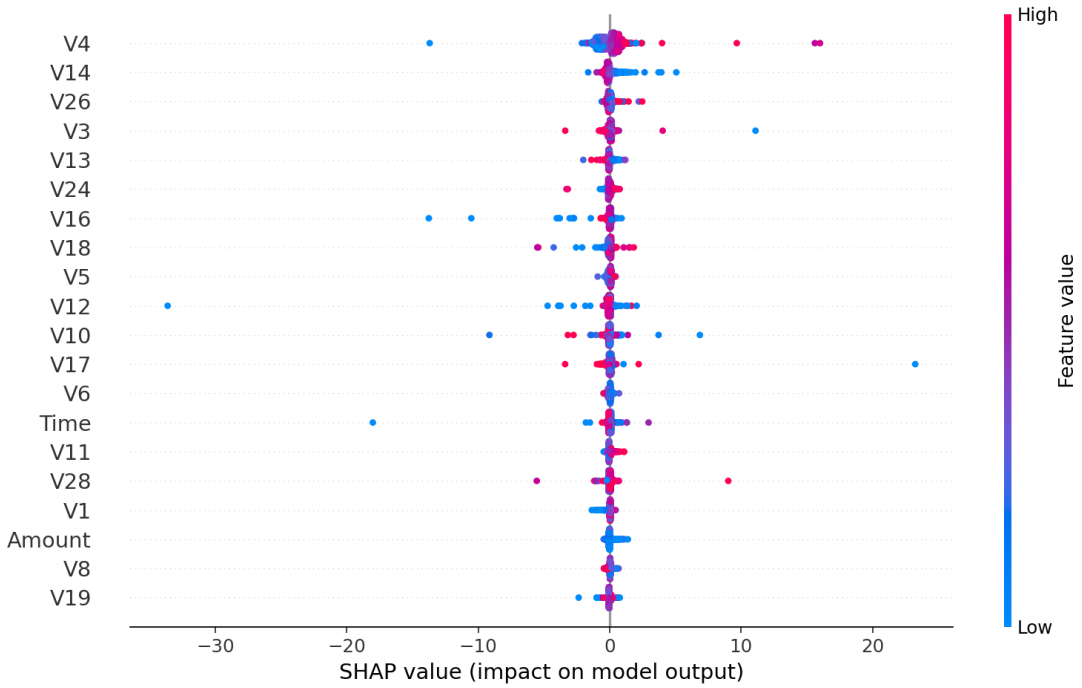


Figure 9: SHAP summary plot from saved artifacts (interpretation aid; not causal proof).

10 Risk Scoring and Decision Logic

The API exposes `risk_score` in the range $[0, 1]$, but this score is explicitly described in the repository as uncalibrated. It should therefore be interpreted as a ranking signal rather than a calibrated fraud probability.

The deployed thresholds are:

- review threshold: `0.7391262534904803`
- high-risk threshold: `0.9999047447184487`

The implemented `risk_tier` logic is:

- `LOW` if `risk_score < threshold_review`
- `REVIEW` if `threshold_review <= risk_score < threshold_high`
- `HIGH` if `risk_score >= threshold_high`

The implemented `decision_recommendation` logic is:

- `ALLOW` for `LOW`
- `STEP_UP_AUTH` for digital-channel `REVIEW`
- `MANUAL_REVIEW` for non-digital-channel `REVIEW`
- `HOLD` for default `HIGH`
- `BLOCK` for `HIGH` when amount is at least 1500.0

Score condition	risk_tier	decision_recommendation
< threshold_review	LOW	ALLOW
>= threshold_review and < threshold_high	REVIEW	STEP_UP_AUTH or MANUAL_REVIEW
>= threshold_high	HIGH	HOLD or BLOCK

Table 2: Score-to-decision mapping.

11 Fraud Detection Workflow

The implemented fraud detection workflow is as follows:

1. A transaction arrives through `POST /predict`.
2. The backend validates the request and extracts the feature vector.
3. The scoring service computes the uncalibrated `risk_score`.
4. The decision service assigns a `risk_tier` and `decision_recommendation`.
5. The reason-code service generates decision-support cues.
6. If the result is `REVIEW` or `HIGH`, the case service creates an `alert` and linked `case`.
7. The transaction enters the fraud alert queue exposed by `GET /alerts` and `GET /cases`.
8. An analyst can move the `case` into `IN_REVIEW`, investigate it, update status, and resolve the outcome.
9. Timeline events record the sequence from scoring to closure.

The implemented `case` lifecycle statuses are `NEW`, `QUEUED`, `IN_REVIEW`, `ESCALATED`, `CONFIRMED_FRAUD`, `FALSE_POSITIVE`, `BLOCKED`, `RELEASED`, and `RESOLVED`. Timeline events include `TRANSACTION_RECEIVED`, `RISK_SCORED`, `FLAGGED`, `ALERT_CREATED`, `CASE_ASSIGNED`, `INVESTIGATION_STARTED`, resolution events, and `CASE_CLOSED`.

Stage	Implemented behavior	Output
Transaction intake	API accepts features and optional transaction context	Validated request
Risk scoring	Model computes uncalibrated `risk_score`	Score in $[0, 1]$
Decision policy	Score mapped to `risk_tier` and `decision_recommendation`	Operational action guidance
Alert creation	`REVIEW` and `HIGH` create `alert` and `case`	Queue item
Investigation	Analyst updates status and reviews reason codes and timeline	Ongoing `case`
Resolution	Analyst marks confirmed fraud, false positive, blocked, released, or resolved	Closed outcome

Table 3: Fraud workflow summary.

12 Backend System Design

The backend is implemented as a FastAPI application. It loads the trained model at startup, instantiates a stream simulator when a model is available, and constructs a case repository through an environment-driven repository factory.

The key backend services are the scoring service, decision service, reason code service, and case service. The core scoring endpoint is `POST /predict`. Supporting endpoints include health, metrics, stream simulation, feature schema access, dataset sampling, alert retrieval, case retrieval, status updates, case resolution, and timeline retrieval.

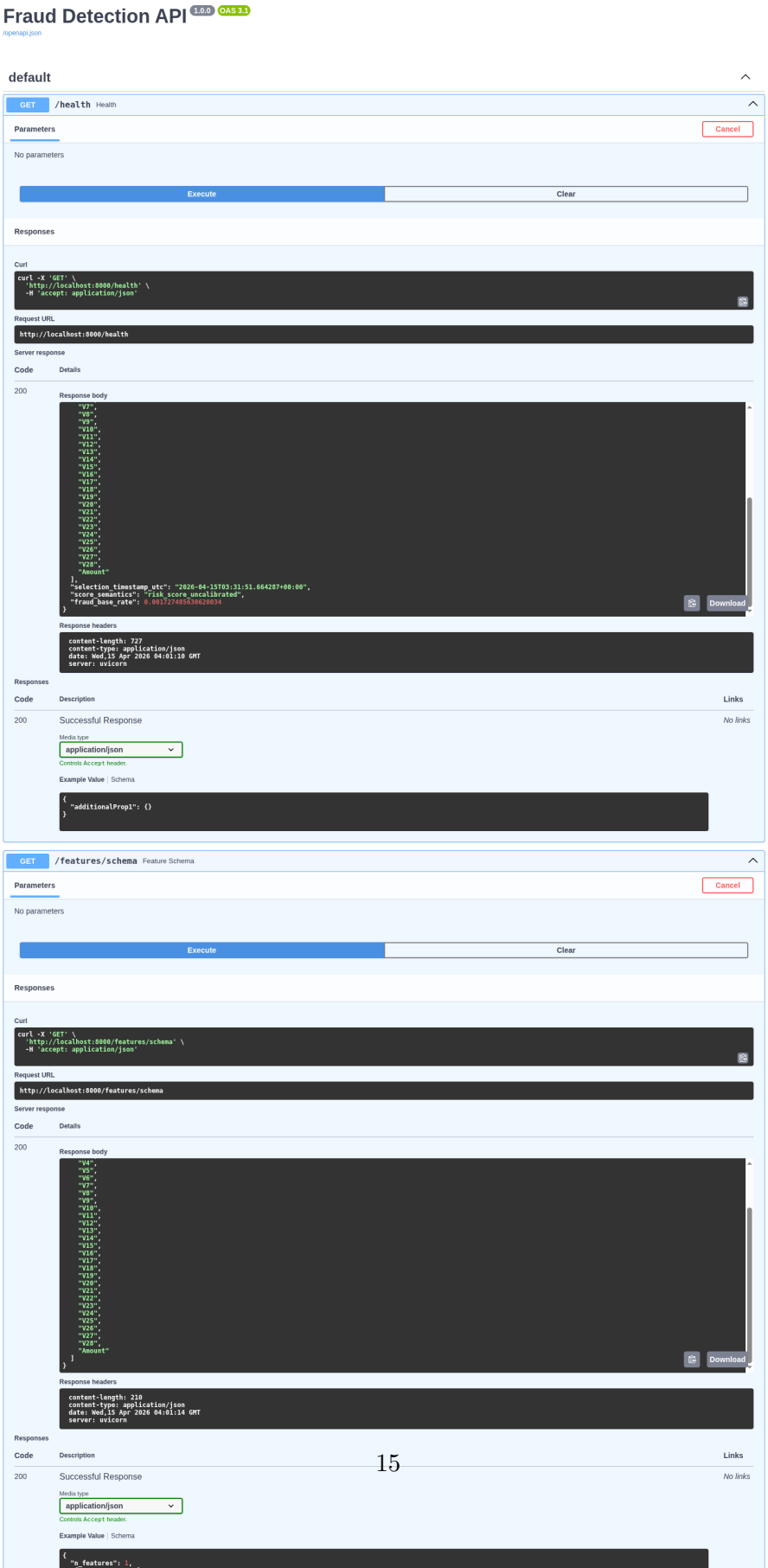
12.1 Core request flow: `POST /predict`

At a high level, the backend converts a prediction request into operational outputs through the following deterministic sequence.

```
request -> validate schema + auth + rate limit
      -> resolve features (features OR features_by_name)
      -> validate feature vector length and numeric finiteness
      -> score_transaction(model, features) -> risk_score
      -> decide_risk_action(risk_score, thresholds) -> risk_tier, action, recommendation
      -> generate_reason_codes(...) -> reason_codes + summary
      -> if risk_tier in {REVIEW, HIGH}: create alert + case + timeline events
      -> record Prometheus metrics + audit event
      -> return PredictResponse(...)
```

Key implementation references: request handling in `src/api/main.py`, scoring in `src/services/scoring_service.py`, decision logic in `src/services/decision_service.py`, workflow orchestration in `src/services/case_service.py`, and persistence in `src/repositories/`.

12.2 API contract evidence



12.3 API endpoint table

Endpoint	Method	Purpose	Status
`/health`	GET	API and model metadata, thresholds, queue size, persistence mode	Implemented
`/features/schema`	GET	Return expected feature contract	Implemented
`/features/random`	GET	Return demo feature vectors	Implemented
`/metrics`	GET	Prometheus metrics export	Implemented
`/predict`	POST	Score transaction and optionally create `alert` and `case`	Implemented
`/stream/pull`	GET	Pull pre-scored demo events	Implemented
`/alerts`	GET	List fraud alerts	Implemented
`/alerts/{alert_id}`	GET	Retrieve alert detail	Implemented
`/alerts/{alert_id}/status`	POST	Update linked case status via alert	Implemented
`/cases`	GET	List cases	Implemented
`/cases/{case_id}`	GET	Retrieve case detail	Implemented
`/cases/{case_id}/status`	POST	Update case lifecycle status	Implemented
`/cases/{case_id}/resolve`	POST	Resolve case outcome	Implemented
`/cases/{case_id}/timeline`	GET	Retrieve investigation timeline	Implemented
`/audit/events`	GET	Retrieve audit events	Implemented, ancillary
`/dataset/samples`	GET	Fetch unlabeled sample rows	Implemented
`/internal/dataset/samples`	GET	Fetch labeled sample rows with token protection	Implemented, internal

For storage, the system supports two repository modes. The in-memory repository is fully implemented and used as the default demo mode. The SQL repository is implemented in code, covered by integration testing, and wired into Docker Compose through PostgreSQL. SQL-backed persistence is configurable and operational when enabled.

13 Frontend System

The frontend is an analyst-facing dashboard implemented in static HTML, CSS, and JavaScript. It connects to the backend API, displays model and threshold metadata, supports live streaming views, and provides a review workflow for suspicious transactions.

The implemented frontend includes a dashboard view for live stream control and summary KPIs, a real-time transaction feed, a fraud alert queue, a review view with case filtering and paging, a case detail panel, and case status actions such as `IN_REVIEW`, `CONFIRMED_FRAUD`, `FALSE_POSITIVE`, and resolution actions.

The frontend also displays a clear score semantics cue stating that the score is uncalibrated rather than a true fraud probability. The UI is therefore more than a visualization layer; it acts as the client for analyst operations against the backend's `alert` and `case` APIs.

13.1 Frontend figures

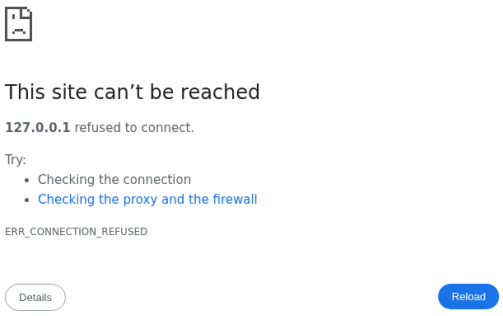


Figure 11: Analyst dashboard view (local deployment evidence screenshot).

14 Monitoring and Observability

The backend exposes Prometheus metrics and includes Grafana dashboard configuration plus MLflow runtime tracking. Monitoring is critical in fraud systems because operational failure can arise from more than model degradation. High latency, queue buildup, false-positive spikes, or unusually low scoring traffic can each undermine fraud operations even when the model artifact itself is unchanged.

The implemented Prometheus metrics include API request volume and status counts, API latency histograms, fraud prediction counts by tier, `risk_tier` distribution, `decision_recommendation` distribution, `alert` and `case` creation counts, `case` status transition counts, `review_queue_size`, confirmed fraud counts, false-positive counts, and risk score aggregates.

The implemented Prometheus alert rules include high 5xx error rate, high p95 API latency, unusually low prediction traffic, unusually high prediction traffic, review queue backlog, and false-positive spike.

MLflow runtime tracking is also implemented. The tracker records runtime traffic metrics such as total requests, predict calls, stream pulls, scored events, average `risk_score`, tier totals, HTTP status families, and decision totals.

14.1 Monitoring figures

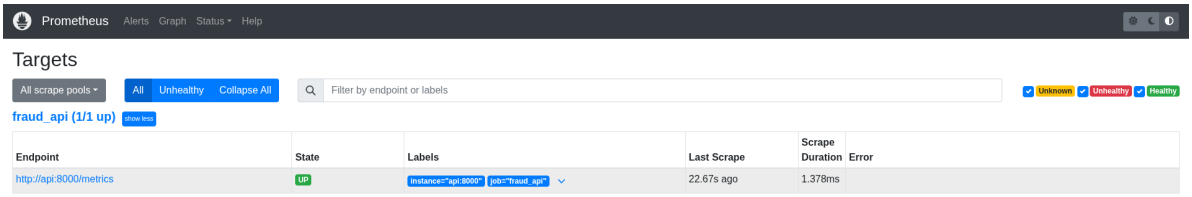


Figure 12: Prometheus scrape targets (deployment evidence screenshot).

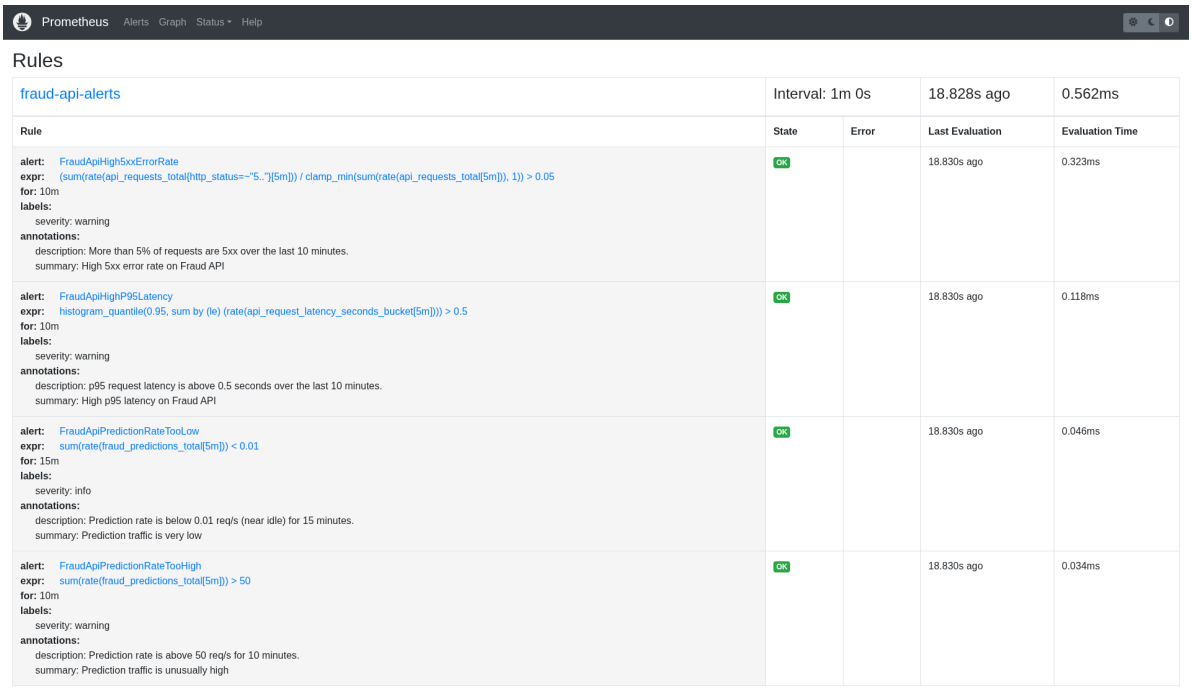


Figure 13: Prometheus alert rules loaded and evaluated (deployment evidence screenshot).

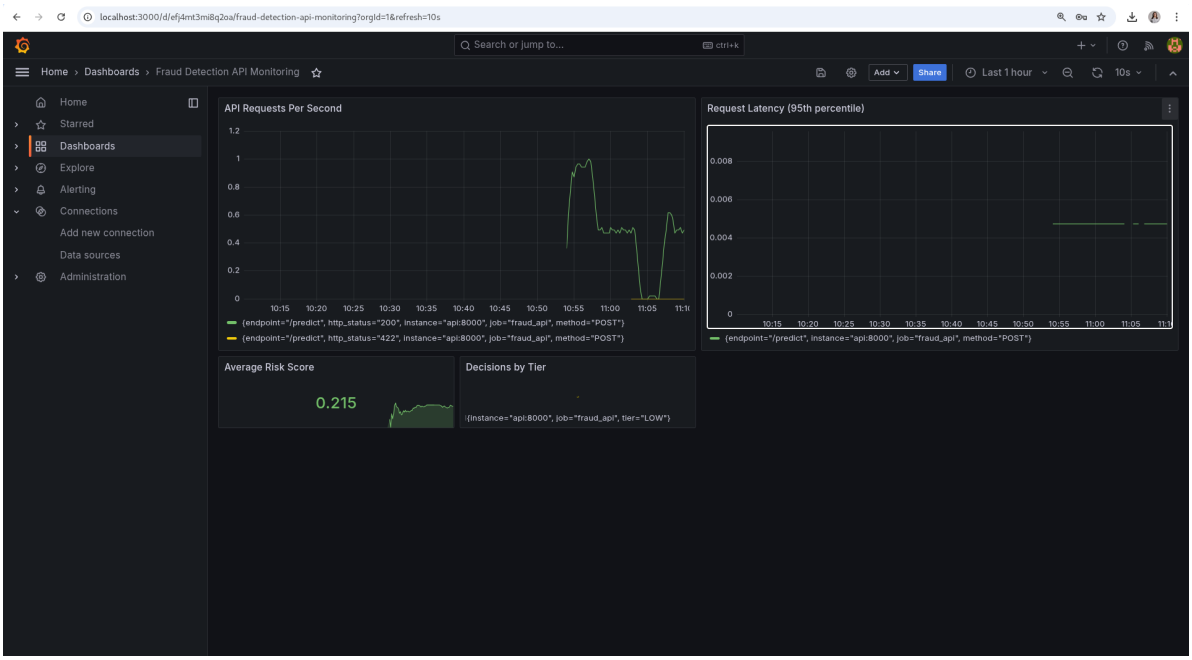


Figure 14: Grafana monitoring dashboard for the fraud API (deployment evidence screenshot).

15 Deployment Architecture

The repository includes a Docker Compose deployment that orchestrates the broader system end to end. The defined services are API, frontend, Prometheus, Grafana, MLflow, and PostgreSQL.

The API container loads the saved fraud model and mounts the artifacts volume. The frontend container serves the analyst dashboard. Prometheus scrapes the API metrics endpoint and loads alert rules. Grafana reads provisioned dashboards. MLflow hosts experiment and runtime tracking. PostgreSQL provides the durable repository option when the API is configured for SQL persistence.

15.1 Docker Compose topology (evidence)

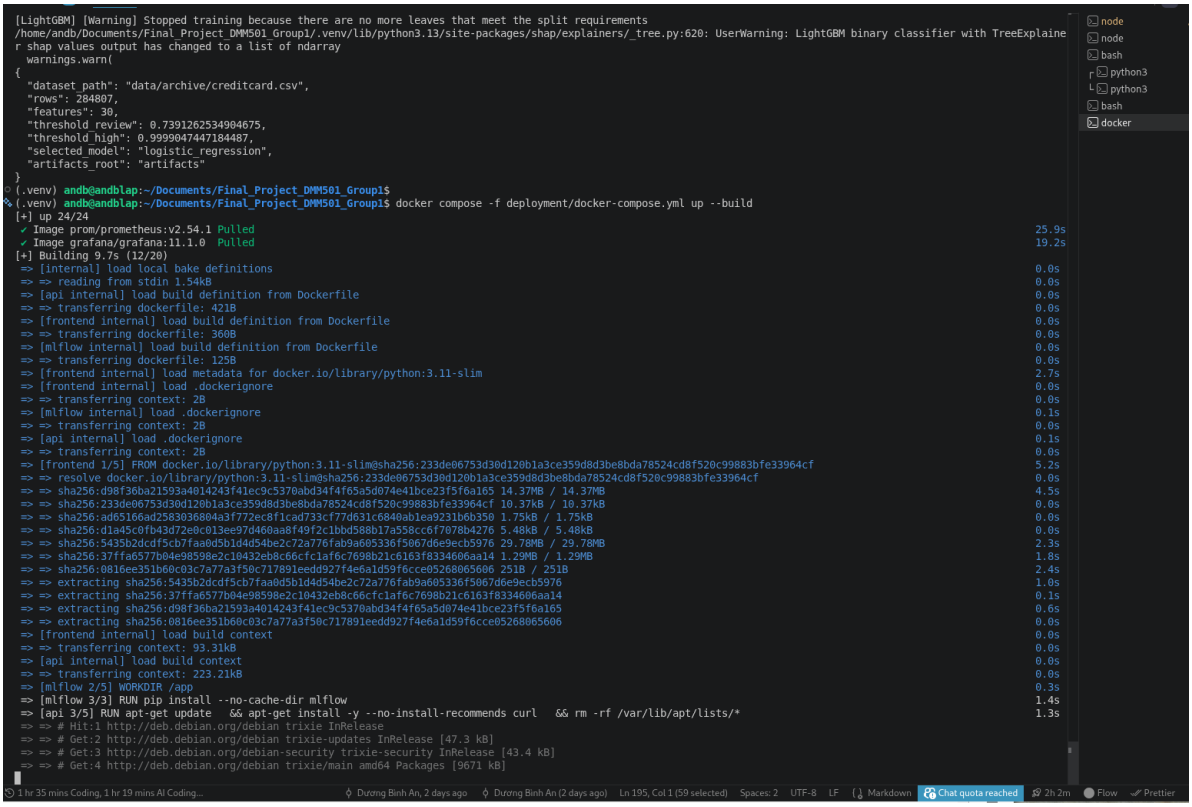


Figure 15: Docker Compose topology screenshot (deployment evidence).

15.2 Local deployment steps

The recommended demo deployment path is Docker Compose:

```
# From repo root:
docker compose -f deployment/docker-compose.yml up --build
```

Key service URLs (default ports):

- API docs: `http://localhost:8000/docs`
- Frontend: `http://localhost:8082/`
- Prometheus: `http://localhost:9090/`
- Grafana: `http://localhost:3000/` (admin password configured by `GF_SECURITY_ADMIN_PASSWORD`)
- MLflow: `http://localhost:5000/`

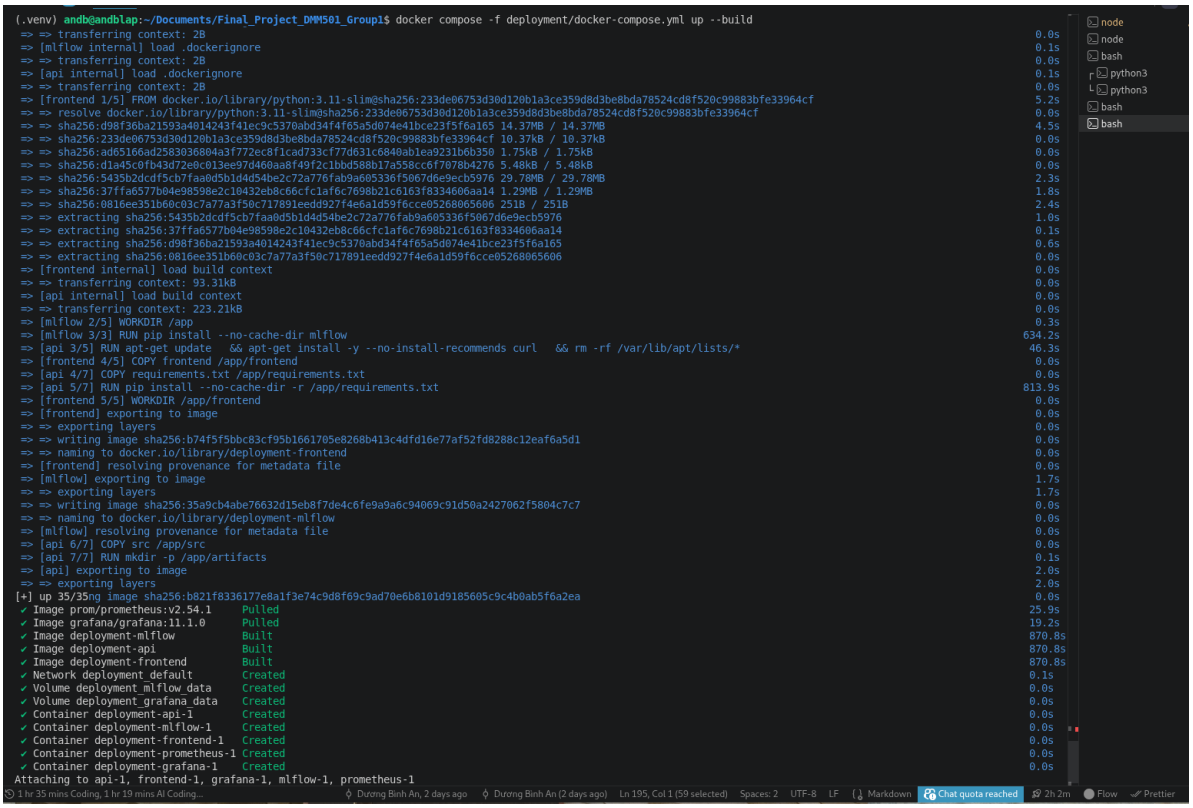


Figure 16: Local `docker compose` up and running services (deployment evidence).

15.3 Deployment configuration (selected environment variables)

The deployment uses environment variables to control model selection, persistence mode, authentication, and monitoring integration. The most important variables are:

Variable	Meaning
MODEL_PATH	Path to the deployed artifact inside the API container (mounted from artifacts/).
MODEL_VERSION	A human-readable identifier returned in /health and responses for traceability.
CASE_REPOSITORY_MODE	memory (demo) or postgres (durable storage).
CASE_DB_URL	SQLAlchemy connection URL for PostgreSQL when postgres is enabled.
CASE_DB_AUTO_MIGRATE	When true, applies DB schema migrations at startup (demo convenience).
API_AUTH_ENABLED	When enabled, requires a token for protected endpoints.
API_TOKENS	Token-to-role mapping used for viewer/analyst/admin authorization.
RATE_LIMIT_ENABLED	Enables request rate limiting middleware.
MLFLOW_RUNTIME_ENABLED	Enables MLflow runtime traffic tracking.
MLFLOW_RUNTIME_TRACKING_URI	MLflow server URI used by the runtime tracker.

Table 5: Selected deployment environment variables.

15.4 CI/CD automation (GitHub Actions)

The repository includes CI workflows for Python tests and container build validation:

- .github/workflows/ci.yml: unit tests, integration tests, and a coverage gate.
- .github/workflows/docker.yml: Docker image builds for API and frontend, plus docker compose config validation.

15.5 System components table

Component	Role	Evidence in repository	Status
Logistic regression model	Fraud scoring	`artifacts/models/final_model.joblib`, `model_info.json`	Implemented
Decision policy service	Map score to tier and recommendation	`src/services/decision_service.py`	Implemented
Reason code service	Produce decision-support cues	`src/services/reason_code_service.py`	Implemented, heuristic
FastAPI backend	Serve scoring and workflow APIs	`src/api/main.py`	Implemented
In-memory repository	Default alert and case storage	`src/repositories/in_memory_case_repository.py`	Implemented, demo-level
SQL/Postgres repository	Durable case storage path	`src/repositories/sql_case_repository.py`	Implemented (configurable)
Frontend dashboard	Analyst queue and case handling UI	`frontend/`	Implemented
Prometheus metrics	Operational observability	`src/monitoring/metrics.py`	Implemented
Grafana dashboards	Visualization layer	`deployment/grafana/`	Implemented
MLflow tracking	Experiment and runtime tracking	`deployment/mlflow/`, `src/monitoring/mlflow_runtime_tracker.py`	Implemented

Table 6: System components and implementation status.

16 Testing and Validation

The repository contains unit, data, integration, security, frontend/API, SQL persistence, and model validation tests. The test inventory includes coverage for preprocessing behavior, request ID generation, dataset path handling and sampling logic, model training artifact creation, model probability range validation, key API endpoints, `alert` and `case` lifecycle flow, token-based authorization, audit events, rate limiting, SQL repository persistence across app restarts, and frontend-to-API interaction.

The repository contains unit, integration, data, model, and frontend/API tests under `tests/`. In the present workspace, integration-style checks pass for multiple API and workflow scenarios, but full-suite runs can encounter Windows temp-directory permission issues affecting tests that use `tmp_path`. This is an environment constraint rather than evidence that the implemented API workflow is absent.

17 Responsible AI

The repository is explicit that the system is a decision-support system and not a fully autonomous fraud adjudication engine. This is the correct framing for banking use.

Three Responsible AI points are especially important: fairness limitations because the dataset does not contain explicit protected attributes; explainability limitations because the core features are PCA-transformed and the reason codes are heuristic rather than causal; and human-in-the-loop design because final outcomes such as `CONFIRMED_FRAUD` and `FALSE_POSITIVE` are produced through `case` resolution, not directly by the model score.

18 Results and Discussion

The project demonstrates that a relatively simple model can perform well on the benchmark dataset when paired with explicit threshold policy and workflow design. The saved artifacts show strong ROC-AUC and meaningful PR-AUC, while the review and high-risk thresholds provide different operating points for queueing and containment.

The system's main strength is integration. It aligns model output, backend orchestration, frontend handling, and monitoring into a coherent fraud decision-support process. The inclusion of `alert` generation, `case` creation, status transitions, timeline events, Prometheus metrics, and Docker deployment makes the repository materially stronger than a standalone notebook or scoring endpoint.

19 Limitations

- Dataset limitation: the Kaggle dataset is a public benchmark and does not represent the full richness of real banking transactions.
- PCA feature opacity: `V1`–`V28` are not business-interpretable features.
- In-memory persistence by default: the primary demo path stores `alert` and `case` records in process memory.
- Security maturity gap: token-based role checks, rate limiting, and audit events exist, but enterprise-grade IAM and hardened RBAC governance are not fully implemented.
- No drift detection: the repository does not implement live data drift, concept drift, or automated retraining triggers.
- No closed feedback loop: resolved fraud outcomes are not yet wired into a model retraining pipeline.

20 Future Work

- Promote the SQL repository path into the default operational mode with durable database-backed `alert`, `case`, and timeline storage.
- Use resolved `case` outcomes as supervised feedback for monitoring and retraining.
- Add scheduled retraining, champion-challenger evaluation, and promotion controls.

- Monitor feature drift, score drift, queue drift, and outcome drift.
- Incorporate interpretable transactional features such as merchant, beneficiary, device, geography, and authentication context.
- Extend current token-role checks into stronger RBAC, identity integration, and hardened audit retention.

21 Conclusion

This repository implements a coherent real-time fraud detection system for banking-style transaction decision support. Its core contribution is the integration of machine learning scoring with operational workflow: the system computes an uncalibrated `risk_score`, maps it to a `risk_tier` and `decision_recommendation`, creates an `alert` and `case` for suspicious transactions, supports analyst investigation, and exposes monitoring and deployment components around that workflow.

The project therefore contributes more than a fraud model. It demonstrates how fraud analytics, backend services, frontend operations, and MLOps tooling can be combined into an end-to-end fraud decision-support system.

Implementation Status Matrix

Capability	Status	Notes
ML scoring API	Implemented	Artifact-backed deployed model and inference path
`risk_tier` and `decision_recommendation` policy	Implemented	Explicit threshold and policy mapping
`alert` and `case` creation	Implemented	Triggered for `REVIEW` and `HIGH`
Investigation timeline	Implemented	Timeline endpoint and event model present
Frontend analyst workflow	Implemented	Dashboard, queue, case actions, and timeline
Prometheus and Grafana monitoring	Implemented	Metrics, rules, and dashboard assets present
MLflow runtime tracking	Implemented	Runtime traffic tracker and deployment service present
SQL/Postgres persistence	Implemented (configurable)	Repository path, migrations, and Compose wiring are available
Reason-code explainability	Partially implemented	Heuristic and policy-derived rather than causal
Streaming behavior	Demo-level	Pull-based simulator, not event-bus-backed banking ingestion
In-memory case persistence	Demo-level	Process-bound storage intended for local demo
Enterprise RBAC/IAM and audit hardening	Future work	Current controls are token-based and limited
Drift detection and automated retraining	Future work	Not implemented
Real banking feature enrichment	Future work	Current data source is benchmark-style and anonymized

Table 7: Implementation status matrix.